

UNIVERSIDADE FEDERAL DE UBERLÂNDIA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
SISTEMA OPERACIONAL

Prof. Marcelo Zanquetta do Nascimento

Sistema Operacional XV6
Implementação do STRF e chamada de sistema getwaittime()

Felipe Fernandes Morotti - 12421BCC063
João Gabriel Ribeiro Viana - 12411BCC099
Giovana de Almeida Faria - 12421BCC023
Mateus Costa Felix - 12411BCC036
Gustavo Henrique Alves Santos - 12411BCC090

ÍNDICE

1. Introdução.....	1
2. Implementação de Código.....	2
2.1) Kernel/proc.h e proc.c.....	2
2.2) Contagem do Tempo de Execução e Espera.....	2
2.3) Chamada de sistema getwaittime().....	3
2.4) Declaração de Burst para o SRTF.....	3
2.5) Implementação do SRTF.....	4
2.6) Programa de teste (waittest.c).....	6
3. Resultados e Análise.....	6
3.1) Comparação com Round Robin.....	8
4. Conclusão.....	9
4.1)Referências.....	10

1.Introdução

O xv6 é um sistema operacional didático que reimplementa o Unix V6 de forma simplificada, utilizado amplamente no ensino de sistemas operacionais. O projeto propôs a extensão do kernel com duas funcionalidades: uma chamada de sistema que retorna o tempo total de espera de um processo (`getwaittime()`) e a substituição do escalonador round-robin original por um escalonador Shortest Remaining Time First (SRTF), uma política preemptiva baseada no menor tempo restante de CPU.

A chamada `getwaittime()` oferece visibilidade sobre o tempo que um processo passa no estado `RUNNABLE` aguardando a CPU, métrica central para avaliar a eficácia do escalonador. O SRTF, por sua vez, deve reduzir o tempo médio de espera, especialmente para processos curtos, mas pode levar à inanição de processos longos.

Este relatório documenta as alterações realizadas no código-fonte, os testes conduzidos, os resultados obtidos (com tabelas e gráficos) e a análise comparativa entre o escalonador original e o SRTF, bem como as limitações encontradas.

2. Implementação de Código

2.1) Kernel/proc.h e proc.c

Foram adicionados quatro campos à struct proc para contabilizar tempo de execução, tempo de espera e o burst de CPU declarado para o SRTF:

```
struct proc {  
    ...  
    int ctime; // tick de criação  
    int rtime; // ticks acumulados em RUNNING  
    int wtime; // ticks acumulados em RUNNABLE (espera)  
    int burst; // burst de CPU esperado, para o SRTF  
};
```

Como esses campos são acessados a partir de outros arquivos além de proc.c, o vetor global de processos precisou ser declarado como extern em proc.h:

```
extern struct proc proc[NPROC];
```

Em allocproc(), no ponto em que os demais campos do processo são inicializados:

```
p->ctime = ticks;  
p->rtime = 0;  
p->wtime = 0;  
p->burst = 0; // 0 = burst ainda não declarado
```

2.2) Contagem do Tempo de Espera

A contabilização foi feita em clockintr() (kernel/trap.c), no mesmo ponto em que o contador global ticks é incrementado. A verificação cpuid() == 0, já existente no xv6, garante que a varredura ocorra uma única vez por tick global, e não uma vez por CPU:

```
void  
clockintr()  
{  
    if (cpuid() == 0) {  
        acquire(&tickslock);  
        ticks++;  
        wakeup(&ticks);  
        release(&tickslock);  
  
        update_time();  
    }  
    w_stimecmp(r_time() + 1000000);  
}
```

A função `update_time()`, adicionada em `proc.c`, percorre a tabela de processos a cada tick e atualiza o campo correspondente ao estado de cada processo:

```
void
update_time(void)
{
    struct proc *p;
    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if (p->state == RUNNING) p->rtime++;
        else if (p->state == RUNNABLE) p->wtime++;
        release(&p->lock);
    }
}
```

A função foi declarada em `kernel/defs.h` para ficar visível em `trap.c`.

2.3) Chamada de Syscall `getwaittime()`

A syscall foi adicionada seguindo o padrão do xv6:

- `kernel/syscall.h`: `#define SYS_getwaittime 23`
- `kernel/syscall.c`: `extern uint64 sys_getwaittime(void)`; e entrada no vetor `syscalls[]`
- `user/user.h`: protótipo `int getwaittime(void)`;
- `user/usys.pl`: `entry("getwaittime")`;
- `kernel/sysproc.c`: implementação

```
uint64
sys_getwaittime(void)
{
    return myproc()->wtime;
}
```

2.4) Declaração de Burst para o SRTF

Para que o SRTF conheça o tempo restante de cada processo, é necessário declarar um burst esperado, já que o sistema não pode prever esse valor. A primeira versão implementada foi a syscall `setburst(int)`, chamada pelo próprio processo logo após o `fork()`:

```
uint64
sys_setburst(void)
{
    int n;
    argint(0, &n);
    if (n < 0) return -1;
```

```

myproc()->burst = n;
return 0;
}

```

Durante os testes, essa abordagem se mostrou problemática: entre o `fork()` e a primeira execução do processo filho, o campo `burst` permanece em seu valor padrão (0), tratado pelo escalonador como "desconhecido". Como todos os filhos recém-criados de um mesmo lote se encontram nessa condição ao mesmo tempo, o SRTF não consegue diferenciá-los na primeira decisão de escalonamento, e o sistema se comporta como FCFS na prática.

A correção foi transferir a responsabilidade de declarar o `burst` para o processo pai, que já está em execução no momento do `fork()`:

```

uint64
sys_setburstpid(void)
{
    int pid, n;
    argint(0, &pid);
    argint(1, &n);
    if (n < 0) return -1;

    struct proc *p;
    int found = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if (p->pid == pid) { p->burst = n; found = 1; }
        release(&p->lock);
    }
    return found ? 0 : -1;
}

```

Com essa mudança, o pai chama `setburstpid(pid_do_filho, burst)` logo após cada `fork()`, de modo que todo processo já nasce com um `burst` definido antes de disputar a CPU pela primeira vez.

2.5) Implementação do SRTF

O escalonador original do xv6 é round-robin: percorre a tabela e despacha o primeiro processo `RUNNABLE` encontrado. A reescrita para SRTF troca esse critério por uma seleção em duas fases, necessária porque o xv6 não permite manter mais de um lock de processo adquirido ao mesmo tempo durante a varredura:

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for (;;) {
        intr_on();

        int found = 0;
        struct proc *chosen = 0;
        int best_remaining = 2147483647;

        for (p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if (p->state == RUNNABLE) {
                int remaining = (p->burst > 0) ? (p->burst - p->rtime) : 2147483647;
                if (chosen == 0 || remaining < best_remaining) {
                    best_remaining = remaining;
                    chosen = p;
                }
            }
            release(&p->lock);
        }

        if (chosen != 0) {
            acquire(&chosen->lock);
            if (chosen->state == RUNNABLE) {
                chosen->state = RUNNING;
                c->proc = chosen;
                swtch(&c->context, &chosen->context);
                c->proc = 0;
                found = 1;
            }
            release(&chosen->lock);
        }

        if (found == 0) {
            intr_on();
            asm volatile("wfi");
        }
    }
}

```

Processos sem burst declarado (burst == 0, caso de init e do shell) recebem prioridade mínima, o que preserva entre eles o comportamento round-robin original.

Dois problemas precisaram ser corrigidos nessa função. O primeiro foi um travamento logo após o início do boot: a condição de seleção usava `remaining < best_remaining` (estritamente menor), e como `init` não possui `burst` declarado, seu `remaining` é igual ao valor-sentinela inicial de `best_remaining`, fazendo a comparação falhar sempre e nenhum processo ser escolhido. A correção foi usar `chosen == 0 || remaining < best_remaining`, garantindo que o primeiro candidato encontrado seja sempre aceito. O segundo problema foi uma chamada indevida a `intr_off()` logo após `intr_on()`, que mantinha as interrupções desabilitadas no momento da instrução `wfi`, impedindo o núcleo de acordar e travando o sistema sempre que não havia processo pronto para executar.

Quanto à preempção, como o timer do `xv6` já força uma nova decisão de escalonamento a cada tick, o comportamento preemptivo do SRTF surge naturalmente da troca de critério de seleção, sem necessidade de lógica adicional.

Uma limitação assumida é que a implementação não trata inanição de processos longos diante de um fluxo contínuo de processos curtos, problema conhecido do SRTF e deixado como trabalho futuro.

2.6) Programa de teste (waittest.c)

Foi criado o programa `user/waittest.c`, adicionado a `UPROGS` no `Makefile`, para validar as duas implementações. Ele possui três modos:

- `waittest solo` — processo único, sem contenção, usado para validar que `getwaittime()` retorna próximo de zero quando não há disputa pela CPU.
- `waittest <n>` — cria `n` processos filhos com `bursts` iguais e não declarados, gerando contenção sob round-robin.
- `waittest srtf <b1> <b2> ...` — cria um filho para cada `burst` informado, chamando `setburstpid()` a partir do pai logo após cada `fork()`, e imprime a ordem de finalização de cada filho junto com seu tempo de espera.

3. Resultados e Análise

Os testes de SRTF foram executados com `make CPUS=1 qemu`, restringindo o sistema a um único núcleo, condição necessária para observar disputa real pela CPU.

Teste 1 — baseline sem contenção:

```
waittest solo
[solo] wait time before=1 after=1 (expect ~0, since nothing else was runnable)
```

O tempo de espera permanece próximo de zero quando não há concorrência, confirmando que `getwaittime()` não acumula tempo indevidamente.

Teste 2 — contenção simples, bursts não declarados:

```
waittest 3
[child 0, pid 5] wait time = 0 ticks
```

```
[child 1, pid 6] wait time = 1 ticks  
[child 2, pid 7] wait time = 2 ticks
```

Com três processos de burst igual disputando a CPU, o tempo de espera cresce de forma consistente com a posição de cada processo na fila, o padrão esperado para uma política sem priorização.

Teste 3 — SRTF com a primeira versão de setburst() (autodeclarada):

```
waittest srtf 50 5 20 3  
[child 0, pid 7, burst 50] wait time = 0 ticks  
[child 1, pid 8, burst 5] wait time = 2 ticks  
[child 2, pid 9, burst 20] wait time = 2 ticks  
[child 3, pid 10, burst 3] wait time = 3 ticks
```

Apesar dos bursts declarados serem 50, 5, 20 e 3, a ordem de finalização seguiu a ordem de criação dos processos, evidenciando o problema descrito na seção 2.4: como nenhum filho tinha burst declarado no momento em que competiu pela primeira vez, todos foram tratados como equivalentes e o escalonador se comportou como FCFS.

Teste 4 — SRTF após a correção (setburstpid, declarado pelo pai):

```
waittest srtf 50 5 20 3  
[child 3, pid 7, burst 3] wait time = 0 ticks  
[child 1, pid 5, burst 5] wait time = 0 ticks  
[child 2, pid 6, burst 20] wait time = 0 ticks  
[child 0, pid 4, burst 50] wait time = 0 ticks
```

Com os filhos criados na ordem 50, 5, 20, 3, a ordem de finalização passou a ser 3, 5, 20, 50, ou seja, estritamente crescente por burst declarado e independente da ordem de criação. Esse resultado é a evidência do funcionamento correto do SRTF.

Nesse último teste, todos os processos reportaram tempo de espera igual a zero. Isso ocorre porque a granularidade do tick do timer é grande em relação à velocidade de execução do burst simulado em waittest.c, de forma que mesmo o processo de burst 50 terminou antes que um número perceptível de ticks em estado RUNNABLE fosse acumulado. Por esse motivo, a ordem de finalização, e não o valor absoluto do tempo de espera, é a evidência mais confiável nesse teste específico.

Resumo comparativo:

Cenário	Escalonador	Ordem de finalização
Bursts 50, 5, 20, 3	Round-robin / FCFS (bug do Teste 3)	50, 5, 20, 3
Bursts 50, 5, 20, 3	SRTF corrigido (Teste 4)	3, 5, 20, 50

O contraste entre os testes 3 e 4, usando exatamente o mesmo conjunto de bursts, mostra tanto a falha de projeto identificada quanto sua correção, sendo essa comparação a evidência mais clara de que o SRTF está funcionando conforme especificado.

3.1) Comparação com Round Robin

Para isolar o efeito da política de escalonamento, o scheduler() foi temporariamente revertido para a versão round-robin original do xv6 (em uma branch git separada), mantendo inalterados todos os demais componentes: getwaittime(), update_time() e setburstpid(). Dessa forma, a única variável entre os dois testes a seguir é o critério de escolha do próximo processo a executar; a instrumentação de tempo permanece idêntica.

O mesmo comando, com os mesmos bursts e a mesma ordem de criação, foi executado nas duas versões, sob CPUS=1:

Round-robin original:

```
waittest srtf 50 5 20 3
[child 1, pid 5, burst 5] wait time = 1 ticks
[child 2, pid 6, burst 20] wait time = 1 ticks
[child 3, pid 7, burst 3] wait time = 2 ticks
[child 0, pid 4, burst 50] wait time = 1 ticks
```

SRTF corrigido:

```
waittest srtf 50 5 20 3
[child 3, pid 7, burst 3] wait time = 0 ticks
[child 1, pid 5, burst 5] wait time = 0 ticks
[child 2, pid 6, burst 20] wait time = 0 ticks
[child 0, pid 4, burst 50] wait time = 1 ticks
```

Sob o round-robin original, a ordem de finalização (5, 20, 3, 50) não guarda relação com o tamanho do burst declarado: o processo de burst 3, o menor de todos, termina apenas em terceiro lugar, atrás de um processo seis vezes maior (burst 20). Os tempos de espera também ficam próximos entre todos os processos (1, 1, 2, 1 ticks), o que é coerente com uma política que trata todos os processos de forma equivalente, alternando turnos igualmente entre eles.

Sob o SRTF corrigido, a ordem de finalização (3, 5, 20, 50) segue estritamente o tamanho do burst declarado, e o processo mais curto deixa de esperar atrás de processos maiores.

Resumo comparativo — round-robin original vs. SRTF:

Escalonador	Ordem de finalização	Tempo (ticks)
Round-robin (original)	5, 20, 3, 50	1, 1, 2, 1
SRTF	3, 5, 20, 50	0, 0, 0, 1

Esse teste completa a análise ao mostrar que a priorização observada no SRTF não é apenas resultado da correção do bug do `setburst()` (comparação já feita entre os testes 3 e 4), mas representa de fato uma mudança de comportamento em relação ao escalonador original do xv6: sob a mesma carga, o round-robin não favorece processos curtos, enquanto o SRTF sim.

4. Conclusão

O trabalho implementou a chamada de sistema `getwaittime()` e o escalonador SRTF no xv6, seguindo o processo padrão de adição de syscalls do sistema e substituindo o critério de seleção do escalonador original sem alterar sua estrutura de sincronização.

Durante o desenvolvimento surgiram três tipos de problema: erros de integração mecânica da syscall, como variáveis não utilizadas, função duplicada e conflito entre cabeçalhos do host e do xv6; uma falha de projeto na declaração de burst, resolvida ao transferir essa responsabilidade do processo filho para o processo pai; e dois erros de lógica no escalonador, uma condição de desempate incorreta e uma inversão indevida de `intr_on()/intr_off()`, ambos causando travamento do sistema no boot.

Os resultados obtidos confirmam que o SRTF prioriza corretamente processos com menor burst declarado, alterando a ordem de finalização em relação ao escalonador original mesmo quando os processos mais curtos são criados por último. A limitação de inanição de processos longos, inerente ao SRTF, foi identificada mas não tratada, ficando como sugestão de trabalho futuro.

4.1)Referências

[1] Cox, R., Kaashoek, M. F., & Morris, R. (2024). xv6: a simple, Unix-like teaching operating system. Draft de 7 de agosto de 2024. Disponível em:

<https://pdos.csail.mit.edu/6.828/2024/xv6.html>.

[2] MIT 6.S081 – Operating System Engineering. Fall 2023. Lab: Scheduling.

<https://pdos.csail.mit.edu/6.S081/2023/labs/scheduling.html>.

[3] Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems. 4th ed. Pearson.